

Aula 01: Fundamentos de Engenharia de Software

Uma introdução aos conceitos essenciais que formam a base da disciplina – do surgimento histórico até os princípios técnicos que guiam o desenvolvimento de software moderno.

ENGENHARIA DE SOFTWARE AULA 01

Olá, meu nome é Eduardo!

Estou como Executivo de Produto e Estratégia Digital na VORA, com mais de 7 anos de experiência em Projetos, Inovação, Tecnologia e Produtos Digitais.

Mestrando em Administração e Marketing na UNICAMP.

Participei de algumas transformações em diversos negócios e mercados.

Meus slides são feitos para vocês explorarem o conteúdo através de ferramentas IA como NotebookLM, gerando dados, videos, infográficos, etc



Como será a dinâmica de nossas aulas?

- Os exercícios são individuais
- Dividiremos a aula em blocos: conteúdo, discussões e exercício prático
- Sempre que possível haverá um Dataset para vocês explorarem
- Recomendo que vocês criem um DECK no NotebookLM do Google
- A nota da Matéria é composta pelo trabalho de conclusão que vale 7 pontos e pela participação em exercícios, que valem 3.
- **Meus slides são feitos com IA, mas com 100% da curadoria feita por mim, slide por slide.**
- Como funcionam os trabalhos quando houver um para entregar?
  - Sempre eu darei um documento completo para exercício do trabalho.
  - Neste exercício sempre haverá espaços para descrer o que vocês produziram.
  - Sempre haverá também transparência sobre os critérios de avaliação.
  - Em cada trabalho há uma folha de resposta para avaliação do trabalho.
  - A maioria dos trabalhos são individuais.



Recursos Essenciais para Vibe Coding

Para mergulhar de vez na prática do Vibe Coding e aproveitar ao máximo os exercícios, o acesso às ferramentas certas é fundamental. Sem elas, a exploração e a experimentação ficam limitadas.

Verifique as opções para obter acesso:

- O **LinkedIn Premium** oferece convênios valiosos, incluindo acesso a plataformas de aprendizado através dos **Premium Perks**.
- Estudantes da **FIAP** possuem uma parceria exclusiva com o **Google**, permitindo a filiação com seu e-mail institucional. Consulte a secretaria da FIAP para saber como ativar seu acesso.

Sobre os Exercícios e o Projeto Final

Para garantir que o aprendizado seja aplicado e avaliado de forma prática, estruturamos os exercícios da seguinte maneira:

Exercícios de Fixação

As atividades realizadas durante as aulas são destinadas exclusivamente à prática e ao aprofundamento do conteúdo. Eles **não terão impacto na sua nota final**.

Projeto Final Avaliativo

O principal desafio que valerá nota será um exercício de automação de uma **Barraca de Praia**. Este projeto integrará os conhecimentos adquiridos.

Prazo de Entrega

Vocês terão um período de **30 dias** para desenvolver e entregar o projeto após a nossa última aula, permitindo tempo para reflexão e execução de qualidade.

Trabalho em Grupo ou Individual

O projeto final pode ser realizado de forma **individual ou em grupos**, proporcionando flexibilidade para escolher a melhor forma de trabalho.



A Engenharia de Prompt e Seu Valor

A Engenharia de Prompt é uma habilidade crucial, especialmente para as nossas aulas de "**Vibe Coding**". Ela permite que vocês comuniquem suas intenções de forma clara e eficaz à inteligência artificial, obtendo resultados mais precisos e relevantes.

Para aprofundar seus conhecimentos, disponibilizamos um material complementar rico em exemplos e técnicas que os ajudarão a dominar esta arte.

O que esperar desta aula?

Para esclarecer as expectativas, esta aula:

- Não é sobre **programação** – você não vai escrever código hoje.
- Não é sobre **ferramentas de IA** – isso virá depois.

Qual é o foco, então?

É uma aula sobre como **pensar o software** e planejar sua estrutura antes de construí-lo. Entender os fundamentos é crucial:

> Um líder que não compreende os **fundamentos de software** delega sem saber o que aprova e avalia entregas sem ter parâmetros claros.

Nossa Jornada de Aprendizado

Ao longo de 4 aulas intensivas, exploraremos os pilares essenciais da Engenharia de Software e mergulharemos em tópicos avançados que moldarão sua visão de desenvolvimento.

01

Conceitos Fundamentais

Começaremos com as bases, garantindo uma compreensão sólida dos princípios que regem a criação de software.

02

Banco de Dados

Dominaremos a arte de armazenar, organizar e recuperar informações de forma eficiente e segura.

03

PRD e PSS

Aprenderemos a documentar produtos (Product Requirements Document) e projetar soluções de software (Project Specification Sheet) com clareza e precisão.

04

Backend e Frontend

Exploraremos a construção de sistemas completos, desde a lógica de servidor até a interface interativa com o usuário.

05

Machine Learning e Hiperpersonalização

Encerraremos com as tendências futuras, entendendo como a IA pode criar experiências de usuário ultra-personalizadas.

Desmistificando o Vibe Coding

O grande dilema de uma aula de **Vibe Coding** é desmistificar que a simples geração de **Prompts** irá, por si só, criar sistemas eficientes e escaláveis. Nossa meta é ir além.

Esta aula tem como objetivo consolidar os **fundamentos de Engenharia de Software**, mas com a velocidade e inovação que o **Vibe Coding** proporciona, sempre com foco em:



Proteção

Garantir sistemas robustos e seguros.



Governança

Estrutura e controle no processo de desenvolvimento.



Design

Soluções bem arquitetadas e intuitivas.



Produtividade

Maximizar resultados com a mínima fricção.



Meu Objetivo com Vocês

Para quem é mais técnico, nosso objetivo é aprofundar os conhecimentos em desenvolvimento de software, explorando as nuances e as melhores práticas da área.

Já para o time de Negócios, a meta é entender os conceitos fundamentais e a engenharia que sustentam a construção de software, permitindo uma visão estratégica mais clara.

Nossos Temas Principais:

- Discovery
- Banco de Dados
- Conceitos de Vibe Coding - PRD-PSS e Blue Print
- Front/End e Back/End
- Hiperpersonalização
- **PLUS - Machine Learning**

Bloco 1 — O que é Software e como ele funciona

O Que é Software?

Software é um conjunto de instruções que diz ao hardware o que fazer. Mas a definição técnica importa menos do que a definição prática: software é a solução de um problema humano expressa em código.

Tipos que o gestor vai encontrar no dia a dia:



Aplicações

O que o usuário toca e vê (app, site, sistema interno).



APIs

O que conecta sistemas entre si (o que você não vê, mas usa o tempo todo).



Plataformas

Infraestrutura onde outros softwares rodam (AWS, Google Cloud).



Modelos de IA

Software que aprende com dados e toma decisões.

> O que diferencia programar de fazer engenharia de software: qualquer pessoa pode escrever código que funciona uma vez; engenharia de software é construir sistemas que funcionam para muitos usuários, por muito tempo, com equipes que mudam.

Tipos de Software

O software se manifesta em formas muito diversas, cada uma com características, desafios e domínios de aplicação distintos.

⚙️ Básico

Apoia outros programas. Ex: sistemas operacionais, compiladores, drivers.

⏱️ Tempo Real

Monitora e controla eventos do mundo físico com restrições rígidas de tempo.

🔬 Científico

Processamento numérico intensivo para simulações, modelos e cálculos de engenharia.

🏢 Empresarial

Sistemas de gestão, ERP, BI e suporte à tomada de decisão corporativa.

🔌 Embarcado

Controla dispositivos e produtos específicos como automóveis, eletrodomésticos e equipamentos médicos.

🤖 Inteligência Artificial

Algoritmos não numéricos para reconhecimento de padrões, aprendizado de máquina e resolução de problemas complexos.

Monolito vs Microserviços — entrega de features

SISTEMA MONOLÍTICO

Um único codebase — features concorrem pelo mesmo ciclo de deploy

Feature	Dev	QA / Deploy	Status
A	S1 → S3 (3 sprints)	S4 ✅	Entregue
B	*Aguarda A* → S5→S6 (2 sprints)	S7 ✅	Bloqueada 4 sem.
C	*Aguarda A e B* → S8 (1 sprint)	S9 ✅	Bloqueada 7 sem.
D	*Aguarda A, B e C* → S10+	S13+ ⚠️	Atrasada indef.

⚠ **Time to market da feature D: 13+ semanas** – bloqueada por dependências sequenciais de deploy.

MICROSERVIÇOS

Cada serviço tem seu próprio ciclo — times e deploys independentes

Feature	Dev	QA / Deploy	Status
A	S1 → S3 (3 sprints)	S4–S5 ✅	Semana 5
B	S1 → S2 (2 sprints)	S3–S4 ✅	Semana 4
C	S1 (1 sprint)	S2–S3 ✅	Semana 3
D	S1 → S3 (3 sprints + teste)	S4–S7 ✅	Semana 8

Feature D entregue na semana 8 – 5 semanas antes do modelo monolítico. Todas as features correm em paralelo.

🏆 Resultado

Microserviços reduzem o **time to market em ~40–60%** ao eliminar dependências sequenciais de deploy e permitir entregas paralelas e independentes por feature.

Sistema Monolítico

Toda a aplicação em um único bloco, fortemente acoplada.

Fortemente acoplado
Componentes dependentes uns dos outros.

Implantação única
Toda a aplicação é implantada junta.

Banco de dados compartilhado
Um único banco para toda a aplicação.

Difícil de escalar
Qualquer mudança exige implantar toda a aplicação.



Microserviços

Aplicação dividida em serviços independentes, cada um com sua própria responsabilidade.



Independência
Cada serviço pode ser desenvolvido, implantado e escalado de forma independente.

Escalabilidade
É possível escalar apenas os serviços que precisam.

Banco de dados próprio
Cada serviço tem seu próprio banco de dados.

Maior flexibilidade
Permite o uso de diferentes tecnologias por serviço.

Bloco 2 — Engenharia de Software na Prática



O Que é Engenharia de Software

A Engenharia de Software é a disciplina que aplica princípios de engenharia à criação de software.

Pense na diferença: um cozinheiro faz um bolo, enquanto uma fábrica produz milhões de bolos idênticos com qualidade controlada. A Engenharia de Software é a **fábrica**.

A Engenharia de Software resolve três problemas essenciais:

Complexidade

Sistemas grandes demais para uma pessoa entender por completo.

Mudança

Requisitos que evoluem enquanto o sistema está sendo construído.

Escala

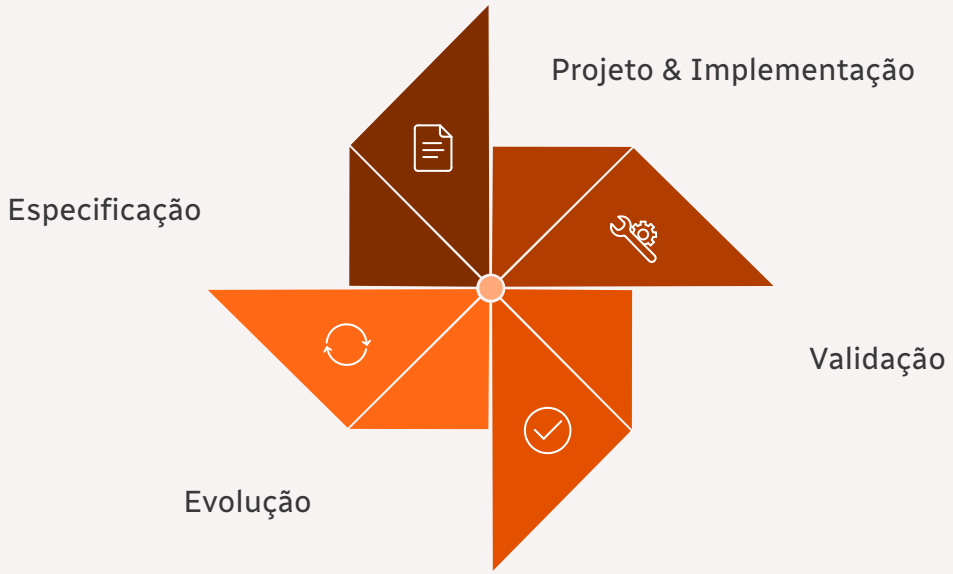
Software que precisa funcionar para 10 ou 10 milhões de usuários.

Para um MBA: Você precisará aprovar projetos, definir prazos, contratar e avaliar equipes. Sem este vocabulário, suas decisões serão no escuro.



Processo de Software

Um processo de software é o conjunto estruturado de atividades, práticas e resultados que conduzem à criação de um produto de software. Embora não exista um processo ideal universal, certas atividades são comuns à maioria das abordagens.



Ferramentas CASE (*Computer-Aided Software Engineering*) automatizam partes desse processo, aumentando a produtividade e a consistência das entregas. A escolha do processo adequado depende do contexto, da equipe e do tipo de produto a ser desenvolvido.

As Grandes Escolhas no Desenvolvimento de Software

No universo do software, engenheiros e líderes de negócios se deparam com decisões arquitetônicas e tecnológicas que moldam o futuro do produto. Não há respostas fáceis; cada escolha envolve uma série de trade-offs.

Monolito vs. Microsserviços

Construir um sistema único e coeso (monolito) ou distribuir as funcionalidades em componentes independentes e menores (microsserviços).

Compilado vs. Interpretado

Linguagens que são totalmente traduzidas antes da execução (compiladas) oferecem performance e verificação rigorosa, enquanto as interpretadas garantem maior flexibilidade.

On-premise vs. Cloud

Manter a infraestrutura em servidores próprios (on-premise) ou delegar essa gestão a provedores externos, utilizando recursos sob demanda na nuvem.

Nenhuma dessas escolhas é universalmente "certa"; todas apresentam trade-offs significativos em custo, velocidade de desenvolvimento, complexidade de gestão e risco operacional.

O papel crucial do líder não é escolher a tecnologia pela tecnologia, mas sim entender as **implicações estratégicas e de negócio** de cada decisão arquitetônica.



IA no Desenvolvimento de Software: Aliada, Obstáculo e Responsabilidade

A Inteligência Artificial já é uma realidade no ciclo de vida do software, atuando como um poderoso copiloto, mas também apresentando desafios e exigindo uma nova camada de responsabilidade gerencial.

Onde a IA Ajuda:

- Geração de código repetitivo
- Sugestão de soluções e padrões
- Detecção proativa de bugs
- Criação e manutenção de testes
- Documentação automática

Onde a IA Atrapalha:

- Dependência excessiva (atrofia do raciocínio crítico)
- Viés nos dados de treinamento (gerando bugs sutis)
- Falsa sensação de segurança (código que parece certo, mas falha)
- Dificuldade na auditoria de decisões complexas

O Papel do Gestor

→ Definir qual código é **crítico** e precisa de revisão humana obrigatória (Zona Vermelha).

→ Não aceitar "a IA fez" como justificativa – **quem entregou é responsável**.

→ Exigir **testes automatizados** rigorosos para todo código gerado por IA.

→ Entender que velocidade de geração **não é igual a qualidade** de entrega.

Bloco 3 — O Problema antes da Solução



A ordem certa — e por que quase todo mundo erra

O erro mais comum no desenvolvimento de software é começar pela solução antes de entender o problema.

O Perigo de Ignorar o Problema

O caso do **Healthcare.gov americano** é um exemplo clássico. Lançado em 2013, custou US$ 2 bilhões e falhou no lançamento. Por quê? Focou em 500 requisitos técnicos detalhados, mas sem um único requisito claro de negócio.

> "O sistema deve ser fácil de usar" **não** é um requisito de negócio claro.

A Pergunta Que Salva Projetos

Antes de escrever a primeira linha de código ou planejar a arquitetura, faça esta pergunta fundamental:

> "Qual decisão de negócio este software vai melhorar?"

Se ninguém na equipe souber responder de forma clara e concisa, o projeto não deve sequer começar. O valor do software está na melhoria que ele traz para o negócio, não na sua existência.

Problema vs. Solução — As Perguntas Certas

Antes de especificar qualquer funcionalidade, a equipe deve responder a questões fundamentais para garantir que o software construído seja a solução certa para o problema certo.

1

Quem tem esse problema?

Um usuário real, com nome e cargo. Não uma persona genérica, mas alguém que você possa observar.

2

Com que frequência esse problema acontece?

A gravidade e a recorrência do problema justificam o investimento no desenvolvimento de software.

3

O que a pessoa faz hoje para contornar o problema sem o sistema?

Entender o "workflow" atual revela dores e oportunidades de melhoria.

4

Como saberemos que o sistema resolveu o problema?

Defina critérios de sucesso mensuráveis para validar a eficácia da solução.

> Um software que resolve um problema mal definido é, tecnicamente, uma solução elegante para o problema errado.

Bloco 4 — Discovery - A porta de Entrada, mas antes vamos falar sobre Requisitos Funcionais e Não Funcionais



O Discovery e os Pilares para um Produto Robusto

O Discovery é uma das partes mais essenciais para desenvolver um software que realmente gere valor. Em nossas aulas, aprofundaremos em três elementos fundamentais:

Product Discovery

A etapa de investigação para **entender o problema e validar a solução**, antes mesmo de começar a construir. É aqui que o "porquê" e o "para quê" são definidos com clareza.

Product-Service System (PSS)

Uma abordagem que considera o produto e os serviços associados de forma integrada, focando na **experiência completa do usuário** e na entrega de valor ao longo do tempo.

Service Blueprint

Um mapa visual detalhado que descreve todo o **processo de serviço de ponta a ponta**, incluindo interações do cliente, pontos de contato e atividades de suporte (frontstage e backstage).

Juntos, estes elementos são cruciais para garantir que seu Produto tenha uma **V1 estável, relevante e orientada a resultados**, minimizando riscos e retrabalhos.

Requisitos Funcionais — o que o sistema faz

Os **Requisitos Funcionais** são as definições essenciais das ações e funcionalidades que um sistema de software deve executar para atender tanto às necessidades do usuário final quanto aos objetivos de negócio. Eles respondem diretamente à pergunta fundamental: "O que o sistema faz?"

A Precisão é Crucial

A clareza e a ausência de ambiguidades na redação dos requisitos funcionais são cruciais para evitar retrabalhos, custos adicionais e falhas na entrega. Um requisito mal formulado pode levar a interpretações diversas e, consequentemente, a um software que não atende à expectativa. Pense neles como o contrato entre a equipe de desenvolvimento e o negócio.

Estrutura Ideal

Um requisito funcional bem escrito segue um padrão específico para garantir que todas as informações necessárias estejam presentes:

> **O sistema DEVE + verbo de ação + complemento + condição quando necessária**

Exemplo Correto

> "O sistema **DEVE** permitir que o usuário autenticado **visualize** o histórico dos últimos 12 meses de transações, **ordenado por data decrescente**."

Este exemplo é específico, mensurável e claro sobre a ação, o ator, o objeto e a condição.

Exemplo Incorreto

> "O sistema **DEVE** mostrar o histórico do usuário."

Este requisito é ambíguo e deixa muitas perguntas sem resposta: Qual histórico? De quanto tempo? Como deve ser mostrado? Para qual usuário?

Exemplos Aplicados de Requisitos Funcionais

Para ilustrar a importância da clareza e da precisão, observe como requisitos funcionais bem definidos se traduzem em capacidades concretas do software, garantindo que o que é construído atenda às expectativas do negócio.



O sistema **DEVE** processar pagamentos via cartão de crédito, débito e Pix, integrando-se com gateways de pagamento seguros.



O sistema **DEVE** enviar notificação por e-mail ao cliente em até 5 minutos após a confirmação bem-sucedida de um pedido.



O sistema **DEVE** permitir que o administrador do sistema desative um usuário, mantendo seus dados para fins de auditoria, mas impedindo novos acessos.



O sistema **DEVE** gerar relatório de vendas consolidado por período selecionado (dia, semana, mês), com opção de exportação nos formatos CSV e PDF.

> A detalhação e a especificidade em cada requisito garantem que não haja espaço para interpretações ambíguas, alinhando a visão de negócio com a execução técnica.

Requisitos Não Funcionais — como o sistema se comporta

Os **Requisitos Não Funcionais (RNFs)** descrevem qualidades, restrições e características do sistema – eles definem como o sistema deve se comportar, e não o que ele faz. São cruciais para a experiência do usuário e para a sustentabilidade do software a longo prazo.

Eles respondem diretamente à pergunta: "como o sistema se comporta?"

As 4 dimensões dos RNFs – os pilares da qualidade de software:



Performance

Velocidade e capacidade de resposta sob carga do sistema.



Disponibilidade

Quanto tempo o sistema precisa estar operacional e acessível.



Segurança

Proteção de dados e controle de acesso contra ameaças.



Usabilidade

Facilidade de uso e intuitividade para o público-alvo.

Exemplos Aplicados de Requisitos Não Funcionais

Os Requisitos Não Funcionais especificam as características de qualidade do sistema, sendo essenciais para a satisfação do usuário e a robustez da aplicação. Observe como cada requisito busca garantir uma experiência de uso superior e a confiabilidade do software.



O sistema deve responder a qualquer consulta em menos de 2 segundos para 95% das requisições com até 5.000 usuários simultâneos.



O sistema deve ter disponibilidade de 99,9% (menos de 9 horas de downtime por ano).



O sistema deve criptografar todos os dados de pagamento em repouso e em trânsito (AES-256).



O sistema deve ser utilizável por uma pessoa sem treinamento prévio, completando o cadastro em menos de 3 minutos.

> Definir claramente os RNFs assegura que o software não apenas funcione, mas que funcione bem, atendendo às expectativas de desempenho, segurança e usabilidade.

RNFs são decisões de negócio — não técnicas

Cada Requisito Não Funcional (RNF) tem um custo associado, e a definição desse custo é uma decisão estratégica do negócio, não apenas uma escolha técnica. As implicações de performance, disponibilidade e escalabilidade se traduzem diretamente em valor e investimento.

Disponibilidade

De **99,9%** para **99,99%**: a diferença é de 8,7 horas/ano para 52 minutos/ano de downtime – e pode significar 10x o custo de infraestrutura.

Tempo de Resposta

De **2 segundos** para **500ms**: pode exigir uma arquitetura completamente diferente, com um custo proporcionalmente maior e mais complexidade.

Escalabilidade

Suportar **1.000 usuários** vs. **100.000**: a escala impõe decisões de arquitetura que afetam todo o projeto desde o início, da infraestrutura ao código.

> A pergunta certa para o gestor em cada RNF é: "quanto vale para o negócio cumprir esse requisito? e quanto custa não cumpri-lo?"

Requisitos Funcionais vs. Não Funcionais — Exemplo Completo

A teoria ganha vida com um exemplo prático. Analise como os Requisitos Funcionais (o que o sistema faz) e os Requisitos Não Funcionais (o como ele se comporta) se manifestam em um sistema de compra de ingressos de cinema.

Requisitos Funcionais

Definem as ações e capacidades essenciais do sistema:

- Exibir filmes em cartaz com sinopse, classificação e horários.
- Permitir seleção visual de assentos em mapa interativo.
- Processar pagamentos (cartão de crédito, débito, Pix).
- Gerar ingresso digital com QR Code após confirmação.
- Permitir login (CPF/senha ou Google).
- Aplicar cupons de desconto e calcular valor final.

Requisitos Não Funcionais

Definem as qualidades e restrições de desempenho e experiência:

- **Performance:** Páginas devem carregar em **< 2s** (conexão 4G).
- **Disponibilidade: 99,9%** de uptime, suportando picos de estreia.
- **Segurança:** Dados de pagamento criptografados; conformidade com PCI-DSS e LGPD.
- **Usabilidade:** Compra completa em **≤ 4 etapas**, funcional em mobile.

> Este exemplo demonstra que um software de sucesso não apenas entrega o que é esperado, mas também o faz com qualidade, segurança e uma ótima experiência de usuário.

Bloco 5 — Agora sim, focar em Discovery

Product Discovery: A Ponte Entre a Ideia e o Sucesso Real

Todo mundo animado, o código rodando, o produto sendo construído – mas alguém parou para perguntar: **estamos construindo a coisa certa?** Discovery é o processo que separa times que criam produtos amados dos que constroem soluções à procura de um problema.

⚠️ O Dilema

Meses de trabalho, dinheiro e energia desperdiçados em algo que ninguém quer. O maior risco não é construir devagar – é construir a coisa errada.

🔍 A Solução

Product Discovery é um processo contínuo de aprendizado e validação que transforma suposições perigosas em evidências concretas e decisões seguras.

🎯 O Resultado

Redução de riscos de valor, usabilidade, execução e viabilidade – culminando em um produto construído com confiança e projetado para causar impacto real.

O que é Discovery — e o que não é

🚫 Não é:

- Pesquisa de mercado
- Brainstorming isolado
- Reunião de alinhamento genérica

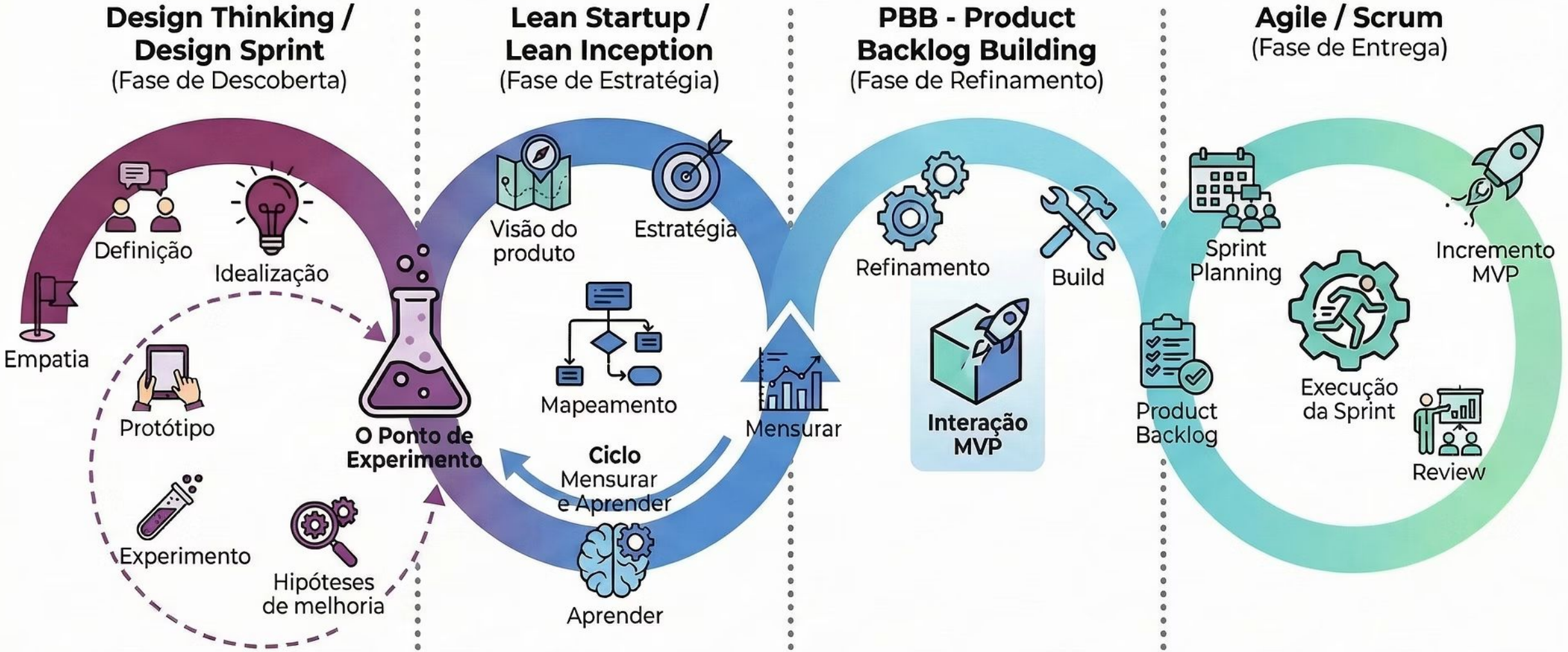
Embora possa incluir elementos dessas atividades, o Discovery vai além de cada uma delas, buscando um propósito mais profundo e estruturado.

✅ É:

O processo sistemático de **reduzir a incerteza** sobre *o que construir* antes de gastar energia e recursos na construção.

É a fase onde as hipóteses são testadas e validadas, transformando "achismos" em conhecimento embasado.

Fluxo Integrado de Desenvolvimento: Do Design Thinking ao Agile



NotebookLM

Definir a Decisão que o Discovery Precisa Apoiar

Antes de mergulhar na investigação, é crucial ter clareza sobre qual decisão fundamental o processo de Discovery busca embasar. Esta etapa define o escopo, o foco e, em última instância, o sucesso de todo o esforço.

1. Qual a Decisão Final?

Ao final do Discovery, qual será a decisão crítica? **Construir X**, **pivotar Y**, ou **descartar Z**? Ter clareza sobre o desfecho desejado evita esforços desperdiçados.

2. Quem Precisa ser Convencido?

Identifique quem patrocina o Discovery e quem são os principais **stakeholders** a serem convencidos pelo resultado. O alinhamento com essas partes é fundamental para a aceitação da decisão final.

3. Prazo é Essencial

Um Discovery sem um **deadline claro** pode se transformar em pesquisa acadêmica interminável. Defina um prazo realista para a tomada da decisão, garantindo foco e agilidade.

4. A Frase de Alinhamento

Documente o objetivo do Discovery em uma frase concisa:

> "Precisamos descobrir se [problema] é real o suficiente para justificar [investimento]."

Isso solidifica o propósito para todos.

5. IA como Facilitadora

Use LLMs como Claude ou ChatGPT para formular a decisão central que o Discovery precisa responder. Descreva o contexto do projeto à IA e peça para ela refinar essa "pergunta" crucial, facilitando o alinhamento.



As Etapas Essenciais do Product Discovery

O Product Discovery não é um evento único, mas um ciclo contínuo de aprendizado e validação. Para transformar incertezas em oportunidades validadas, siga estas etapas fundamentais:

Definir o Problema e Objetivo

Alinhar o desafio central a ser resolvido e o que se espera alcançar com o processo de Discovery.

Gerar Hipóteses de Solução

Com base no problema e nos dados iniciais, formular ideias e suposições de como resolvê-lo.

Pesquisar e Coletar Evidências

Validar ou invalidar as hipóteses através de pesquisas com usuários, análise de dados e experimentos.

Sintetizar e Tomar Decisões

Analisar os aprendizados, refinar o entendimento e decidir sobre o próximo passo: o que construir, o que pivotar ou o que descartar.

Cada etapa alimenta a próxima, garantindo que o produto desenvolvido esteja alinhado com as necessidades reais dos usuários e os objetivos estratégicos de negócio.

Etapa 01 - ALINHAR O PROBLEMA *"O que estamos tentando descobrir?"*

Estruturar o Problem Statement

Um **Problem Statement** bem definido é a bússola que guia todo o processo de Product Discovery. Ele garante que todos no time compreendam o cerne do desafio e foquem na resolução das necessidades mais críticas do usuário, em vez de se perderem em funcionalidades aleatórias.

Para que seja eficaz, deve:

- Descrever **quem tem o problema** (o segmento de usuário).
- Explicitar **qual é o problema** que ele enfrenta.
- Justificar **por que o problema importa** (qual o impacto para o usuário e para o negócio).
- Indicar **como saberíamos que o problema foi resolvido** (métricas de sucesso).

É crucial que seja específico o suficiente para guiar o desenvolvimento de soluções, mas sem prescrever a solução em si. Evite problem statements genéricos como "usuários querem economizar tempo", pois isso não oferece direcionamento claro.

> Um formato útil para estruturar o Problem Statement é:
>
> "Pessoas que **[contexto]** precisam de uma forma de **[necessidade]** porque **[insight]**, mas hoje **[barreira atual]**."

Dica de IA: Utilize Large Language Models (LLMs) como Claude ou ChatGPT. Cole briefings, e-mails, atas de reunião e documentos internos relevantes no LLM e peça para que ele redija o Problem Statement seguindo este formato estruturado. Isso pode agilizar o alinhamento e garantir uma formulação concisa e completa.



Gerar as Perguntas Essenciais do Discovery

Após definir o problema e mapear as suposições, o próximo passo é formular as perguntas certas. Elas são o motor do Discovery, direcionando a pesquisa e os testes para validar ou invalidar as hipóteses mais arriscadas.

1

Perguntas que Decidem

Concentre-se em questões cujas respostas capacitem a equipe a tomar decisões com confiança. Cada pergunta deve direcionar a validação ou invalidação de uma hipótese chave do Problem Statement.

2

Priorize pela Urgência

Identifique quais perguntas não respondidas representam o maior risco para o projeto. Estas são as hipóteses críticas que precisam ser testadas e respondidas primeiro para mitigar incertezas.

3

IA como Geradora

Alimente seu Problem Statement em um LLM (Claude, ChatGPT). Peça para gerar as 10 perguntas mais críticas que necessitam de resposta antes de qualquer decisão, agilizando a criação da sua agenda de Discovery.

4

Ferramenta Útil

Utilize ferramentas como Claude ou ChatGPT como seu **gerador de agenda de Discovery**. Eles podem refinar e organizar as perguntas, transformando um brainstorming em um plano de ação estruturado.



Formular Hipóteses Sólidas com o Apoio da IA

A formulação de hipóteses é o cerne do Discovery, transformando o que aprendemos em declarações testáveis. A Inteligência Artificial pode potencializar esse processo, tornando-o mais rigoroso e abrangente.

Estruturar Achados em Hipóteses

Converta os insights coletados do Discovery em afirmações claras e testáveis sobre o usuário, o problema e a possível solução que a IA pode oferecer.

Avaliar o Risco da Hipótese

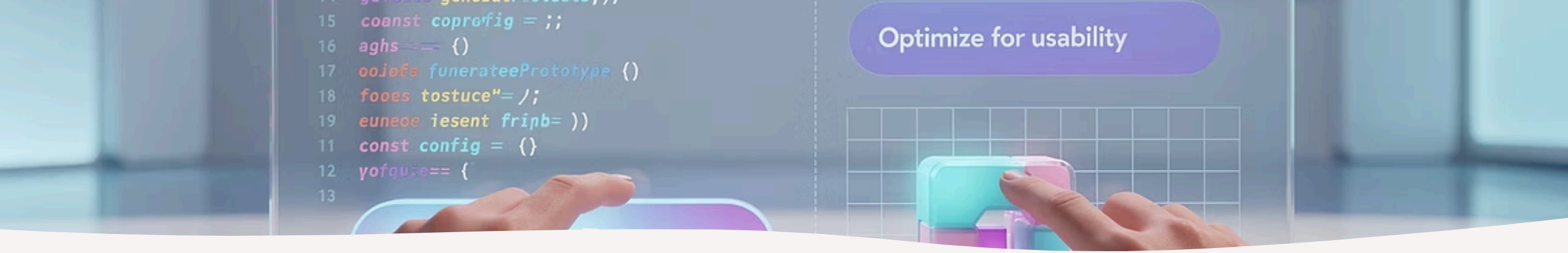
Priorize as hipóteses com base na sua incerteza (o quão pouco sabemos) e no impacto potencial de estarem erradas, focando nas que podem invalidar sua solução.

O Advogado do Diabo (IA)

Utilize um LLM para "advogar contra" cada hipótese, apontando falhas lógicas ou cenários alternativos. Isso força a equipe a considerar pontos fracos e fortalecer suas proposições.

Descobrir Novas Hipóteses

Peça a LLMs para gerar hipóteses adicionais ou complementares, explorando ângulos que a equipe pode ter negligenciado, ampliando as possibilidades de solução.



Gerar protótipos funcionais em horas

A prototipagem rápida é fundamental no Discovery para validar ideias sem o custo e tempo de desenvolvimento completo. Com o apoio da IA, podemos transformar descrições em linguagem natural em protótipos navegáveis em tempo recorde, acelerando o ciclo de feedback.

Descritivo Natural para Protótipo

Descreva a solução em linguagem natural e obtenha um protótipo navegável sem escrever código manual.

Iteração por Linguagem Natural

Itere o protótipo descrevendo os ajustes e refinamentos em linguagem natural, sem editar código diretamente.

Apoio de IA na Geração

Utilize ferramentas de IA para gerar o código-fonte da interface diretamente a partir das suas descrições.

Ferramentas Potenciais

Explore plataformas como **v0.dev**, **Bolt.new** ou **Cursor** para a geração de interfaces funcionais e interativas.